

C# 基礎課程 03

Bill Chung
V2026.1 台北夏季班

請尊重講師的著作權及智慧財產權!

Build School 課程之教材、程式碼等、僅供課程中學習用、請不要任意自行散佈、重製、分享，謝謝

遞增與遞減運算子

理解以下的流程

- `++i`
- `i++`
- `--i`
- `i--`
- 它們都是遞增（`++`）或遞減（`--`），但究竟有何不同。

提示

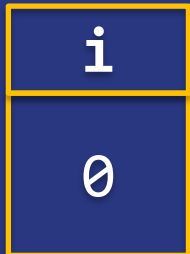
- C# 程式碼遇到指派運算子，會先完成右方的流程，才執行指派運算子

先記住以下關於 i++ 的程式碼

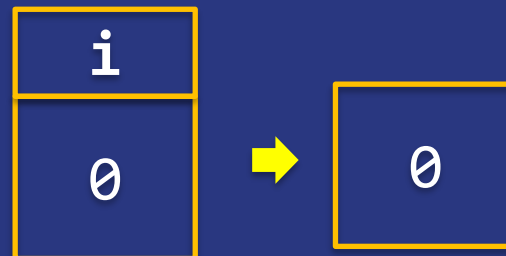
```
static void Main(string[] args)
{
    int i = 0;
    i = i++;
    Console.WriteLine(i);
    Console.ReadLine();
}
```

DoubleSamples\DoubleSample001

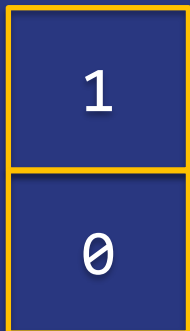
(1) 記憶體裡面有個變數，
名稱是 **i**，值為 **0**



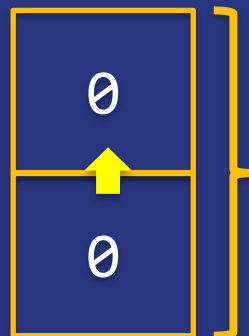
(2) 電腦將**i**變數的值複製一份，
放在記憶體的另一個區塊



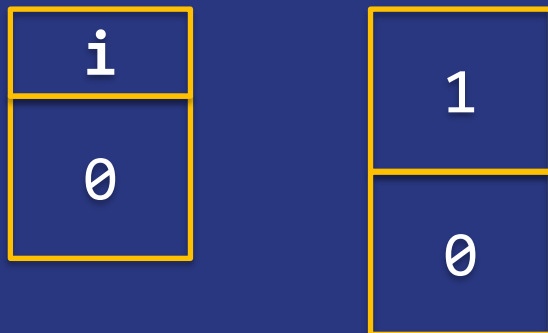
(4) 電腦將上面那個 **0 + 1**



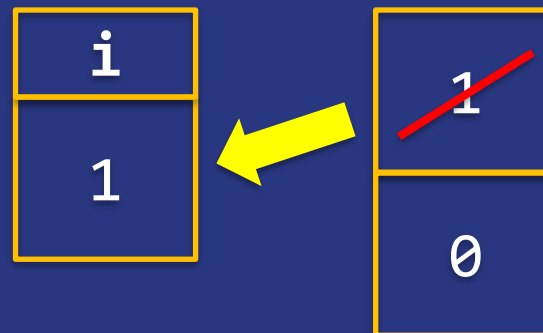
(3) 電腦將剛剛的**0**再度複製，
再放在另一個區塊



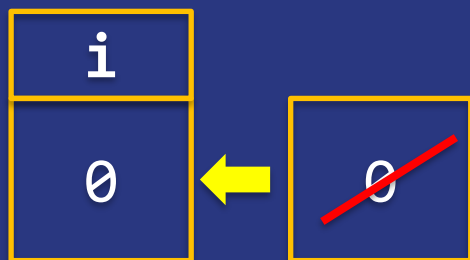
(5) 此時記憶體是這樣的



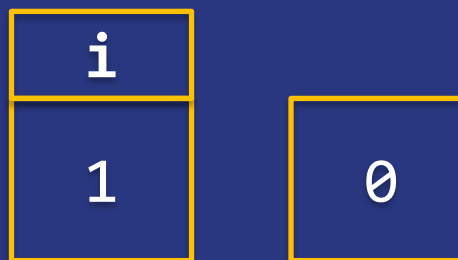
(6) 電腦將上方的1複製給i變數
並且將上方的1移除



(8) 電腦將剩下的那個0複製給
i變數，並將那個0移除



(7) 此時記憶體是這樣，完成了
指派運算子右方的運算式

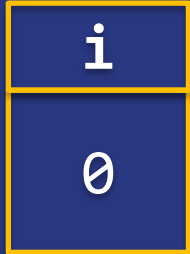


先記住以下關於 ++i 的程式碼

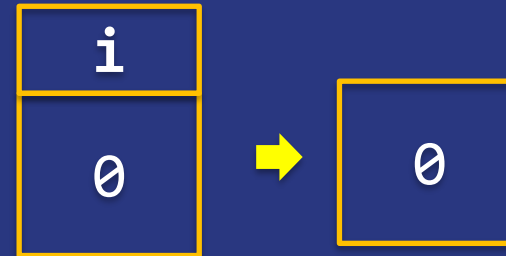
```
static void Main(string[] args)
{
    int i = 0;
    i = ++i;
    Console.WriteLine(i);
    Console.ReadLine();
}
```

DoubleSamples\DoubleSample002

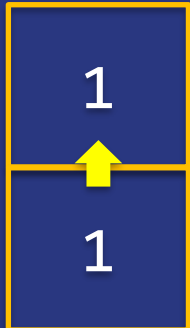
(1) 記憶體裡面有個變數，
名稱是 **i**，值為 **0**



(2) 電腦將**i**變數的值複製一份，
放在記憶體的另一個區塊



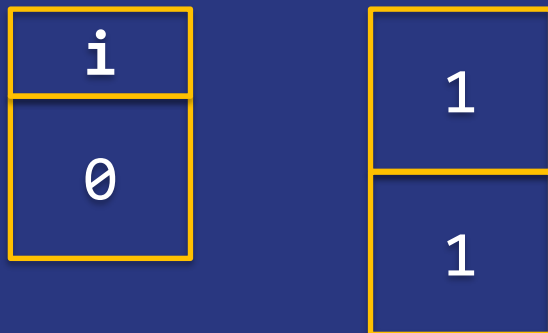
(4) 電腦將剛剛的**1**再度複製，
再放在另一個區塊



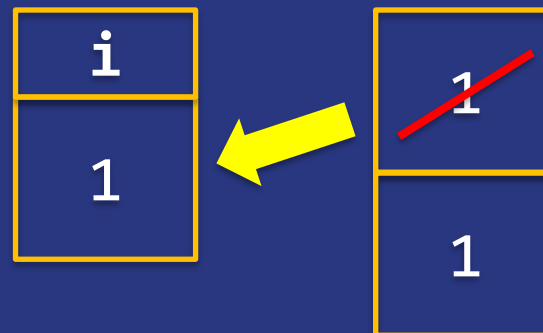
(3) 電腦將剛剛的**0+1**



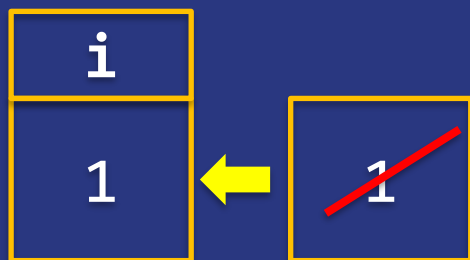
(5) 此時記憶體是這樣的



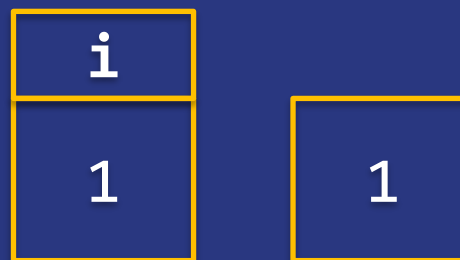
(6) 電腦將上方的1複製給i變數
並且將上方的1移除



(8) 電腦將剩下的那個1複製給
i變數，並將那個1移除



(7) 此時記憶體是這樣，完成了
指派運算子右方的運算式



討論

- 經由上述的分析，你們是否可以順利地解釋 $i=i++$ 和 $i=++i$ 的不同？



linq

linq 簡介

- 「查詢」(Query) 是一種從資料來源擷取資料的運算式， 而且使用專用的查詢語言來表示。
- 一段時間之後，針對不同類型的資料來源已開發出不同的語言，例如 SQL 是用於關聯式資料庫，而 XQuery 則是 XML。 因此，開發人員必須針對他們所需支援的資料來源類型或資料格式學習新的查詢語言。

- LINQ 提供一致的模型來使用各種資料來源和格式的資料，從而簡化此情況。在 LINQ 查詢中，您所處理的一定是物件。不論您要查詢及轉換的資料是存在 XML 文件、SQL 資料庫、ADO.NET 資料集，還是 .NET 集合，以及其他任何有可用 LINQ 提供者 (Provider) 的格式中，都是使用相同的基本程式碼撰寫模式。
- 請參考 [語言整合式查詢 \(LINQ\)](#)

LINQ to Object

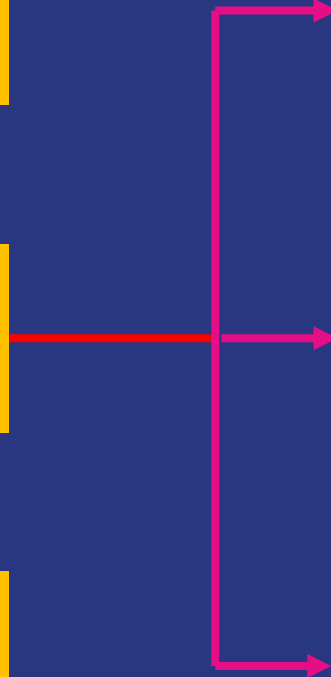
LINQ to ADO.NET

LINQ to XML

LINQ to DataSet

LINQ to SQL

LINQ to
Entities



Enumerable Class

- [文件參考](#)
- Linq to object 的實作在這個類別中
- 這些方法都是【擴充方法】
- 【擴充方法】也是一種語法糖，它使得靜態方法可以像執行個體方法一般的使用。(只有像而已)

Linq 的敘述的種類

- Query Expression
 - 類似 SQL 查詢的寫法
- Method Expression
 - 以 C# Method 的形式呈現

Query Expression 入門

from <某個東西>

in <來源 or 另一個 query expression >

<

<where expression> or

<group expression> or

<join expression>

>

<select expression>

LAB

基本的 Query Expression

新增一個方案及專案

- 方案名稱：LinqSamples
- 專案名稱：LinqSample001
- 範本：Console Application
- csproj nullable disable

加入 MyData Class，並建立其內容

```
internal class MyData
{
    public string Name
    { get; set; }

    public int Age
    { get; set; }
}
```

在 Program Class 加入產生 List<MyData> 的方法
並且在 Main Method 中呼叫它以產生一個集合

```
static void Main(string[] args)
{
    List<MyData> list = CreateList();
}
static List<MyData> CreateList()
{
    return new List<MyData>()
    {
        new MyData { Name = "Bill", Age = 47 },
        new MyData { Name = "John", Age = 37 },
        new MyData { Name = "Tom", Age = 48 },
        new MyData { Name = "David", Age = 36 },
        new MyData { Name = "Bill", Age = 35 },
    };
}
```

參考範例有另一種 target-typed new expression 寫法

在 Main Method 加入 Query Expression 和 顯示結果的程式碼

```
static void Main(string[] args)
{
    List<MyData> list = CreateList();

    //Query Expression
    IEnumerable<MyData> people =
        from data in list
        where data.Name == "Bill"
        select data;

    //顯示結果
    foreach (MyData person in people)
    {
        Console.WriteLine($"{person.Name} 是 {person.Age} 歲");
    }
    Console.ReadLine();
}
```




執行

討論

- 請和你的夥伴們討論 Query Expression 的運作



var 宣告

- 強型別
- 隱含型別 (所以這是個語法糖)
- 或稱右(後)決議型別
- 只能做為宣告區域變數使用

討論

- 請開啟 VarSample 範例，和你的同學討論裡面的程式碼。
- 務必了解 var 怎麼用



LAB

將剛剛 Query Expression 轉成
Method Expression 的寫法

在 LinqSamples 加入新專案

- 方案名稱：LinqSamples
- 專案名稱：LinqSample002
- 範本：Console Application
- csproj nullable disable

加入 MyData Class，並建立其內容

```
internal class MyData
{
    public string Name
    { get; set; }

    public int Age
    { get; set; }
}
```

在 Program Class 加入產生 List<MyData> 的方法
並且在 Main Method 中呼叫它以產生一個集合

```
static void Main(string[] args)
{
    List<MyData> list = CreateList();
}
static List<MyData> CreateList()
{
    return new List<MyData>()
    {
        new MyData { Name = "Bill", Age = 47 },
        new MyData { Name = "John", Age = 37 },
        new MyData { Name = "Tom", Age = 48 },
        new MyData { Name = "David", Age = 36 },
        new MyData { Name = "Bill", Age = 35 },
    };
}
```


C# 9.0 後可以使用 target-typed new expression

```
return new()  
{  
    new() { Name = "Bill", Age = 47 },  
    new() { Name = "John", Age = 37 },  
    new() { Name = "Tom", Age = 48 },  
    new() { Name = "David", Age = 36 },  
    new() { Name = "Bill", Age = 35 },  
};
```

在 Main Method 加入 Method Expression 和 顯示結果的程式碼

```
static void Main(string[] args)
{
    var list = CreateList();

    // Method Expression
    var people = list.Where((x) => x.Name == "Bill");

    //顯示結果
    foreach (MyData person in people)
    {
        Console.WriteLine($"{person.Name} 是 {person.Age} 歲");
    }

    Console.ReadLine();
}
```

執行

Select 哪裡去了？

- 在剛剛這麼簡單的查詢下，Method Expression 的 Select 是可以省略的。若要包含 Select 語法則需寫成以下型式

```
var people = list.Where((x) => x.Name == "Bill").Select((x) => x);
```

找到第一個符合條件的資料

- **First**

- 找到符合條件的第一個，沒找到就會發生例外

- **FirstOrDefault**

- 找到符合條件的第一個，沒找到就回傳預設值
- **.NET 6.0** 新增自訂預設值

預設值的概念

- 對於 `byte`, `short`, `int`, `long`, `float`, `double`, `decimal` 這些數值型的資料，預設值是 `0`。
- 對於 `bool` 預設值是 `false`。
- 對於 `string`, `object` 或其他參考型別的資料，預設值是 `null`。
- `null` 是一種特殊的資料，代表甚麼都沒有。

LAB

First 與 FirstOrDefault

在 LinqSamples 加入新專案

- 方案名稱：LinqSamples
- 專案名稱：LinqSample003
- 範本：Console Application
- csproj nullable disable

加入 MyData Class，並建立其內容

```
internal class MyData
{
    public string Name
    { get; set; }

    public int Age
    { get; set; }
}
```

在 Program Class 加入產生 List<MyData> 的方法
並且在 Main Method 中呼叫它以產生一個集合

```
static void Main(string[] args)
{
    var list = CreateList();
}
static List<MyData> CreateList()
{
    return new List<MyData>()
    {
        new MyData { Name = "Bill", Age = 47 },
        new MyData { Name = "John", Age = 37 },
        new MyData { Name = "Tom", Age = 48 },
        new MyData { Name = "David", Age = 36 },
        new MyData { Name = "Bill", Age = 35 },
    };
}
```

在 Main Method 加入 程式碼

```
static void Main(string[] args)
{
    // 開始用 var 了
    var list = CreateList();
    // 這裡的 person1 是單個物件，也就是 MyData person1
    var person1 = list.FirstOrDefault((x) => x.Age < 37);
    Console.WriteLine($"小於 37 歲的人第一個被找到的是 : {person1.Name}");

    // 如果找不到，就會回傳自訂的物件 (.NET 6.0 新增)
    var customDefault = list.FirstOrDefault((x) => x.Age < 30,
                                              new MyData { Name = "Not Found", Age = 0 });
    Console.WriteLine($"找不到，所以回傳自訂物件 : {customDefault.Name}");

    // 因為找不到，就會跳出例外
    var person2 = list.First((x) => x.Age < 30);
    Console.WriteLine($"小於 30 歲的人第一個被找到的是 : {person2.Name}");

    Console.ReadLine();
}
```

執行

找到最後一個符合條件的資料

- **Last**

- 找到符合條件最後一個，沒找到就會發生例外

- **LastOrDefault**

- 找到符合條件的最後一個，沒找到就回傳預設值
- **.NET 6.0** 新增自訂預設值

LAB

Last 與 LastOrDefault

在 LinqSamples 加入新專案

- 方案名稱：LinqSamples
- 專案名稱：LinqSample004
- 範本：Console Application
- csproj nullable disable

加入 MyData Class，並建立其內容

```
internal class MyData
{
    public string Name
    { get; set; }

    public int Age
    { get; set; }
}
```


在 Program Class 加入產生 List<MyData> 的方法
並且在 Main Method 中呼叫它以產生一個集合

```
static void Main(string[] args)
{
    var list = CreateList();
}
static List<MyData> CreateList()
{
    return new List<MyData>()
    {
        new MyData { Name = "Bill", Age = 47 },
        new MyData { Name = "John", Age = 37 },
        new MyData { Name = "Tom", Age = 48 },
        new MyData { Name = "David", Age = 36 },
        new MyData { Name = "Bill", Age = 35 },
    };
}
```

在 Main Method 加入 程式碼

```
static void Main(string[] args)
{
    var list = CreateList();
    // 這裡的 person1 是單個物件，也就是 MyData person1
    var person1 = list.LastOrDefault((x) => x.Age > 35);
    Console.WriteLine($"大於 35 歲的人最後一個被找到的是 : {person1.Name}");

    // 如果找不到，就會回傳自訂的物件 (.NET 6.0 新增)
    var customDefault = list.LastOrDefault((x) => x.Age > 50,
                                           new MyData { Name = "Not Found", Age = 130 });
    Console.WriteLine($"找不到，所以回傳自訂物件 : {customDefault.Name}");

    // 因為找不到，就會跳出例外
    var person2 = list.Last((x) => x.Age > 50);
    Console.WriteLine($"大於 50 歲的人最後一個被找到的是 : {person2.Name}");

    Console.ReadLine();
}
```



執行

找到符合條件而且是唯一一個的資料

- **Single**

- 找到符合條件而且是唯一一個，沒找到就會發生例外
- 如果符合條件的結果有兩個(含)以上，發生例外

- **SingleOrDefault**

- 找到符合條件而且是唯一一個，沒找到就回傳預設值
- 如果符合條件的結果有兩個(含)以上，發生例外
- **.NET 6.0** 新增自訂預設值

LAB

Single 與 SingleOrDefault

在 LinqSamples 加入新專案

- 方案名稱：LinqSamples
- 專案名稱：LinqSample005
- 範本：Console Application
- csproj nullable disable

加入 MyData Class，並建立其內容

```
internal class MyData
{
    public string Name
    { get; set; }

    public int Age
    { get; set; }
}
```

在 Program Class 加入產生 List<MyData> 的方法
並且在 Main Method 中呼叫它以產生一個集合

```
static void Main(string[] args)
{
    var list = CreateList();
}
static List<MyData> CreateList()
{
    return new List<MyData>()
    {
        new MyData { Name = "Bill", Age = 47 },
        new MyData { Name = "John", Age = 37 },
        new MyData { Name = "Tom", Age = 48 },
        new MyData { Name = "David", Age = 36 },
        new MyData { Name = "Bill", Age = 35 },
    };
}
```


在 Main Method 加入 程式碼

```
static void Main(string[] args)
{
    var list = CreateList();
    // 這裡的 person1 是單個物件，也就是 MyData person1
    var person1 = list.SingleOrDefault((x) => x.Name == "Tom");
    Console.WriteLine($"找到唯一的 : {person1.Name}");

    // 如果找不到，就會回傳自訂的物件 (.NET 6.0 新增)
    var customDefault = list.SingleOrDefault((x) => x.Name == "Not Found",
        new MyData { Name = "Not Found", Age = 0 });
    Console.WriteLine($"找不到，所以回傳自訂物件 : {customDefault.Name}");

    // 因為找不到唯一（裡面有兩個 Bill） 就會跳出例外
    var person2 = list.Single((x) => x.Name == "Bill");
    Console.WriteLine($"找到唯一的 : {person2.Name}");

    Console.ReadLine();
}
```

註：SingleOrDefault 若是遇到有兩個符合條件也會跳出例外



執行

討論

- 討論 First、Last 和 Single 的差異
- 討論 xxxOrDefault 的作用



正確使用 xxxOrDefault

- 不論是 FirstOrDefault、LastOrDefault 或任何其他 xxxOrDefault 都有可能會回傳 null。
- 該怎麼處理這樣的情形？

LAB

正確處理預設值

在 LinqSamples 加入新專案

- 方案名稱：LinqSamples
- 專案名稱：LinqSample006
- 範本：Console Application
- csproj nullable disable

加入 MyData Class，並建立其內容

```
internal class MyData
{
    public string Name
    { get; set; }

    public int Age
    { get; set; }
}
```

在 Program Class 加入產生 List<MyData> 的方法
並且在 Main Method 中呼叫它以產生一個集合

```
static void Main(string[] args)
{
    var list = CreateList();
}
static List<MyData> CreateList()
{
    return new List<MyData>()
    {
        new MyData { Name = "Bill", Age = 47 },
        new MyData { Name = "John", Age = 37 },
        new MyData { Name = "Tom", Age = 48 },
        new MyData { Name = "David", Age = 36 },
        new MyData { Name = "Bill", Age = 35 },
    };
}
```


在 Main Method 加入 程式碼 一定找不到李小龍

```
static void Main(string[] args)
{
    var list = CreateList();
    var person = list.FirstOrDefault((x) => x.Name == "李小龍");
    // 判斷回傳結果是否為 null
    if (person == null )
    {
        //如果是 null 則另行處理
        Console.WriteLine("查無此人");
    }
    else
    {
        Console.WriteLine($"找到 : {person.Name} - {person.Age}");
    }

    Console.ReadLine();
}
```

執行

**如果有可能出现 null
一定要處理**

取出某個位置的資料

- **ElementAt**

- 找到特定位置的資料，沒找到就會發生例外

- **ElementAtOrDefault**

- 找到特定位置的資料，沒找到就回傳預設值
 - **.NET 6.0** 新增自訂預設值

在 LinqSamples 加入新專案

- 方案名稱：LinqSamples
- 專案名稱：LinqSample007
- 範本：Console Application
- csproj nullable disable

加入 MyData Class，並建立其內容

```
internal class MyData
{
    public string Name
    { get; set; }

    public int Age
    { get; set; }
}
```

在 Program Class 加入產生 List<MyData> 的方法
並且在 Main Method 中呼叫它以產生一個集合

```
static void Main(string[] args)
{
    var list = CreateList();
}
static List<MyData> CreateList()
{
    return new List<MyData>()
    {
        new MyData { Name = "Bill", Age = 47 },
        new MyData { Name = "John", Age = 37 },
        new MyData { Name = "Tom", Age = 48 },
        new MyData { Name = "David", Age = 36 },
        new MyData { Name = "Bill", Age = 35 },
    };
}
```

在 Main Method 加入 程式碼

```
static void Main(string[] args)
{
    int index = 1;
    var list = CreateList();
    // 這裡的 person 是單個物件，也就是 MyData person
    var person = list.ElementAtOrDefault(index);
    if (person == null)
    {
        Console.WriteLine("查無此人");
    }
    else
    {
        Console.WriteLine($"找到索引為 : {index} 的人是 {person.Name}");
    }
    Console.ReadLine();
}
```




執行

判斷是否有符合條件的資料

- **Any**

- 判斷是否有一個或以上數量的資料符合條件

- **All**

- 判斷是否所有的資料都符合條件

LAB

Any

在 LinqSamples 加入新專案

- 方案名稱：LinqSamples
- 專案名稱：LinqSample008
- 範本：Console Application
- csproj nullable disable

加入 MyData Class，並建立其內容

```
internal class MyData
{
    public string Name
    { get; set; }

    public int Age
    { get; set; }
}
```

在 Program Class 加入產生 List<MyData> 的方法
並且在 Main Method 中呼叫它以產生一個集合

```
static void Main(string[] args)
{
    var list = CreateList();
}
static List<MyData> CreateList()
{
    return new List<MyData>()
    {
        new MyData { Name = "Bill", Age = 47 },
        new MyData { Name = "John", Age = 37 },
        new MyData { Name = "Tom", Age = 48 },
        new MyData { Name = "David", Age = 36 },
        new MyData { Name = "Bill", Age = 35 },
    };
}
```

在 Main Method 加入 程式碼

```
static void Main(string[] args)
{
    var list = CreateList();
    string name = "David";
    bool result = list.Any((x) => x.Name == name);
    if (result)
    {
        Console.WriteLine($"找到了 : {name}");
    }
    else
    {
        Console.WriteLine($"找不到 : {name}");
    }
    Console.ReadLine();
}
```



執行

LAB

All

在 LinqSamples 加入新專案

- 方案名稱：LinqSamples
- 專案名稱：LinqSample009
- 範本：Console Application
- csproj nullable disable

加入 MyData Class，並建立其內容

```
internal class MyData
{
    public string Name
    { get; set; }

    public int Age
    { get; set; }
}
```

在 Program Class 加入產生 List<MyData> 的方法
並且在 Main Method 中呼叫它以產生一個集合

```
static void Main(string[] args)
{
    var list = CreateList();
}
static List<MyData> CreateList()
{
    return new List<MyData>()
    {
        new MyData { Name = "Bill", Age = 47 },
        new MyData { Name = "John", Age = 37 },
        new MyData { Name = "Tom", Age = 48 },
        new MyData { Name = "David", Age = 36 },
        new MyData { Name = "Bill", Age = 35 },
    };
}
```

在 Main Method 加入 程式碼

```
static void Main(string[] args)
{
    var list = CreateList();
    string name = "Bill";
    bool isAllBill = list.All((x) => x.Name == name);
    if (isAllBill)
    {
        Console.WriteLine($"全都是 {name}");
    }
    else
    {
        Console.WriteLine($"有些人不叫 {name}");
    }
    bool isAllOverForty = list.All((x) => x.Age >= 40);
    if (isAllOverForty)
    {
        Console.WriteLine("大家都超過 40 歲");
    }
    else
    {
        Console.WriteLine("有人不到 40 歲");
    }
    Console.ReadLine();
}
```

執行

運算

- **Sum**
 - 加總
- **Min**
 - 取得最小值
- **Max**
 - 取得最大值
- **Count**
 - 取得數量
- **Average**
 - 取得平均值

LAB

使用 linq 運算

在 LinqSamples 加入新專案

- 方案名稱： LinqSamples
- 專案名稱： LinqSample010
- 範本：Console Application
- csproj nullable disable

加入 MyData Class，並建立其內容

```
internal class MyData
{
    public string Name
    { get; set; }

    public int Age
    { get; set; }
}
```

在 Program Class 加入產生 List<MyData> 的方法

```
static List<MyData> CreateList()
{
    /* 使用 集合運算式 */
    return
    [
        new() { Name = "Bill", Age = 47 },
        new() { Name = "John", Age = 37 },
        new() { Name = "Tom", Age = 48 },
        new() { Name = "David", Age = 36 },
        new() { Name = "Bill", Age = 35 }
    ];
}
```

這次改用 C#12.0 的集合運算式寫法

在 Main Method 加入 程式碼

```
static void Main(string[] args)
{
    var list = CreateList();
    // 計算 list 中, 所有 Age 的總和
    int total = list.Sum((x) => x.Age);
    Console.WriteLine($"年齡的總和為: {total}");
    // 取得 list 中, Age 最小的值
    var minAge = list.Min((x) => x.Age);
    Console.WriteLine($"最小的年齡為 : {minAge}");
    // 取得 list 中, Age 最大的值
    var maxAge = list.Max((x) => x.Age);
    Console.WriteLine($"最大的年齡為 : {maxAge}");
    // 取得 list 中的數量
    //請注意 Count 和 Count() 是不一樣的
    int count = list.Count();
    Console.WriteLine($"list 總個數為 : {count}");
    int countOfBill = list.Count((x) => x.Name == "Bill");
    Console.WriteLine($"list 中的 Bill 總數量為 : {countOfBill}");
    // 取得所有年齡的平均值
    var average = list.Average((x) => x.Age);
    Console.WriteLine($"年齡的平均值為 : {average}" );
    Console.ReadLine();
}
```



執行

LAB

複合查詢
這個練習做有條件的 Min , Sum 與
Average

在 LinqSamples 加入新專案

- 方案名稱： LinqSamples
- 專案名稱： LinqSample011
- 範本：Console Application
- csproj nullable disable

加入 MyData Class，並建立其內容

```
internal class MyData
{
    public string Name
    { get; set; }

    public int Age
    { get; set; }
}
```


在 Program Class 加入產生 List<MyData> 的方法

```
static List<MyData> CreateList()
{
    /* 使用 集合運算式 */
    return
    [
        new() { Name = "Bill", Age = 47 },
        new() { Name = "John", Age = 37 },
        new() { Name = "Tom", Age = 48 },
        new() { Name = "David", Age = 36 },
        new() { Name = "Bill", Age = 35 }
    ];
}
```

C#12.0 的集合運算式

在 Main Method 加入 程式碼

```
static void Main(string[] args)
{
    var list = CreateList();
    // 找出名稱為 Bill 中的最小 Age
    var min = list.Where((x) => x.Name == "Bill").Min((x) => x.Age);
    Console.WriteLine($"所有 Bill 中最小的年齡是 : {min}");

    // 計算名稱為 Bill 的年齡總和
    var total = list.Where((x) => x.Name == "Bill").Sum((x) => x.Age);
    Console.WriteLine($"所有 Bill 的年齡總和是 : {total}");

    var average = list.Where((x) => x.Name == "Bill").Average((x) => x.Age);
    Console.WriteLine($"所有 Bill 的年齡平均是 : {average}");

    Console.ReadLine();
}
```



執行

討論

- 是否了解如何使用複合的查詢方式呢？



新版 MinBy/MaxBy

- MinBy
- MaxBy
- .NET 6 新增
- 語法更簡潔
- 回傳整個物件

LAB

MinBy and MaxBy

在 LinqSamples 加入新專案

- 方案名稱： LinqSamples
- 專案名稱： LinqSample012
- 範本：Console Application
- csproj nullable disable

加入 MyData Class，並建立其內容

```
internal class MyData
{
    public string Name
    { get; set; }

    public int Age
    { get; set; }
}
```


在 Program Class 加入產生 List<MyData> 的方法

```
static List<MyData> CreateList()
{
    /* 使用 集合運算式 */
    return
    [
        new() { Name = "Bill", Age = 47 },
        new() { Name = "John", Age = 37 },
        new() { Name = "Tom", Age = 48 },
        new() { Name = "David", Age = 36 },
        new() { Name = "Bill", Age = 35 }
    ];
}
```

C#12.0 的集合運算式

在 Main Method 加入 程式碼

```
static void Main(string[] args)
{
    var list = CreateList();
    MyData minObject_before = list.First(x => x.Age == list.Min(x => x.Age));
    Console.WriteLine($"最小年齡的人是 : {minObject_before.Name},{minObject_before.Age}");
    MyData minObject_after = list.MinBy((x) => x.Age);
    Console.WriteLine($"最小年齡的人是 : {minObject_after.Name},{minObject_after.Age}");

    MyData maxObject_before = list.First(x => x.Age == list.Max(x => x.Age));
    Console.WriteLine($"最大年齡的人是 : {maxObject_before.Name},{maxObject_before.Age}");
    MyData maxObject_after = list.MaxBy((x) => x.Age);
    Console.WriteLine($"最大年齡的人是 : {maxObject_after.Name},{maxObject_after.Age}");

    Console.ReadLine();
}
```

執行

聯集、交集與差集

- **Union**

- 取得兩個 `IEnumerable <T>` 的聯集

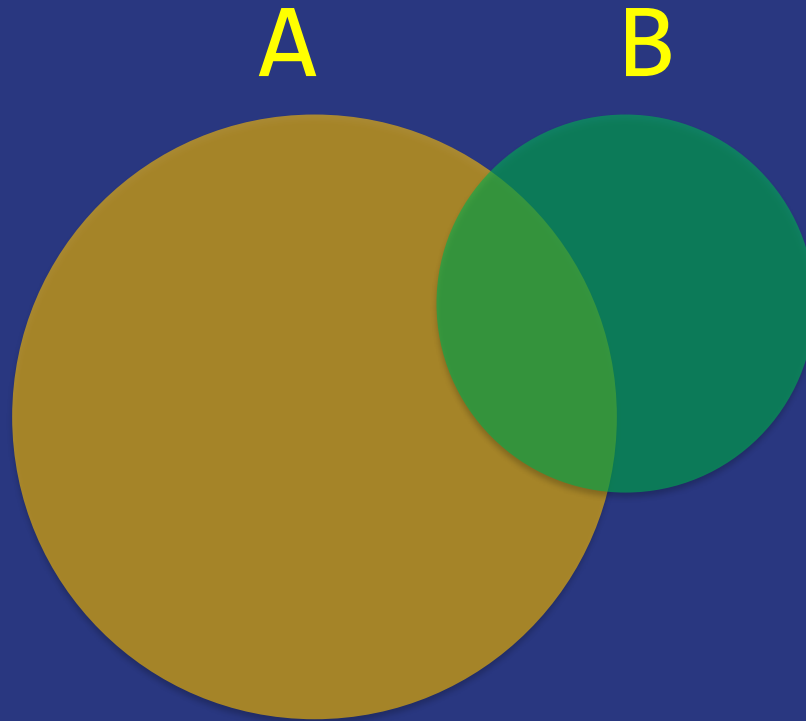
- **Intersect**

- 取得兩個 `IEnumerable <T>` 的交集

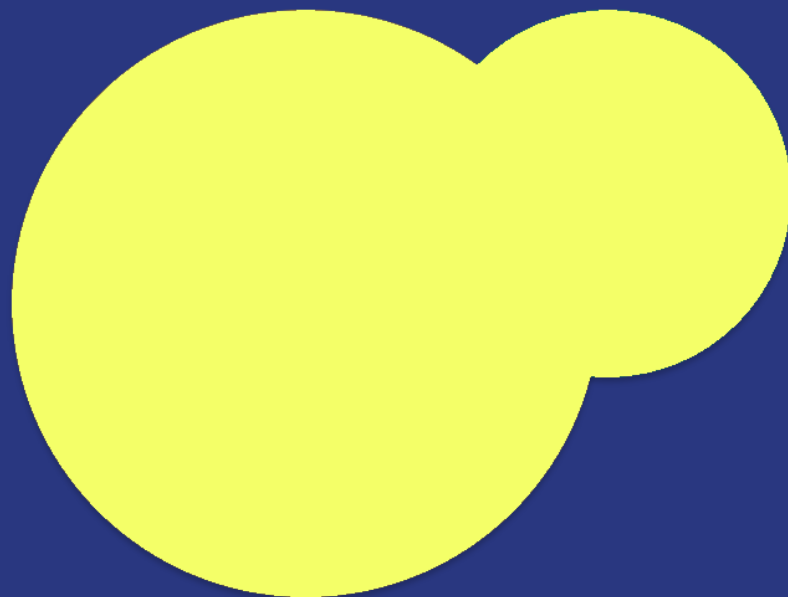
- **Except**

- 取得兩個 `IEnumerable <T>` 的差集
- 【A 差集 B】 和 【B 差集 A】是不一樣的

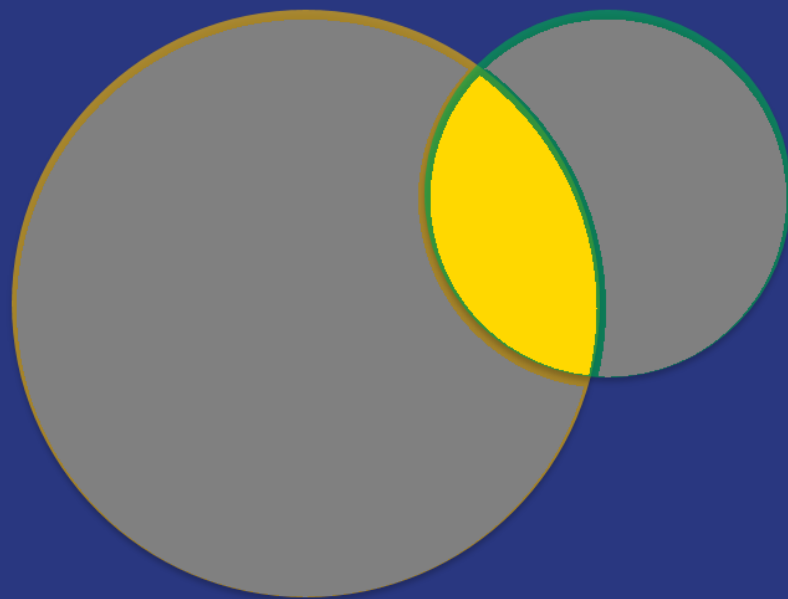
假設有兩個 `IEmuerable<T>`



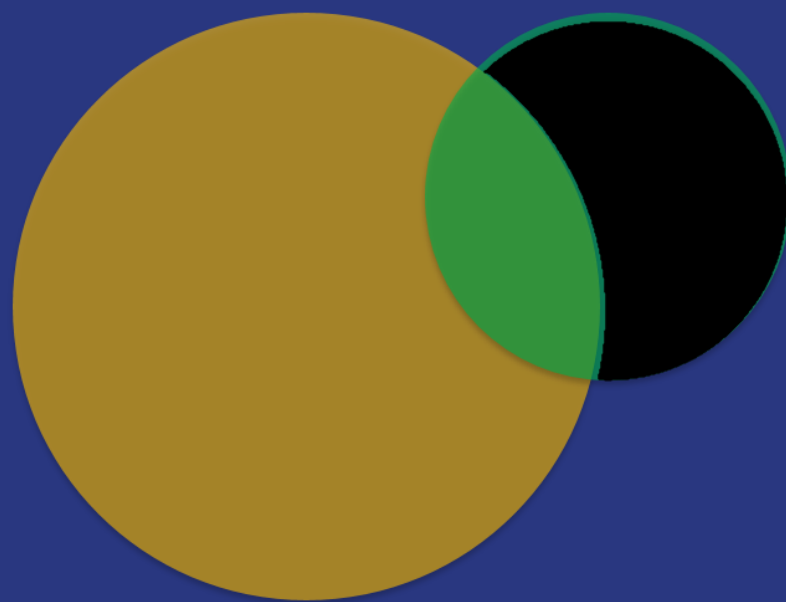
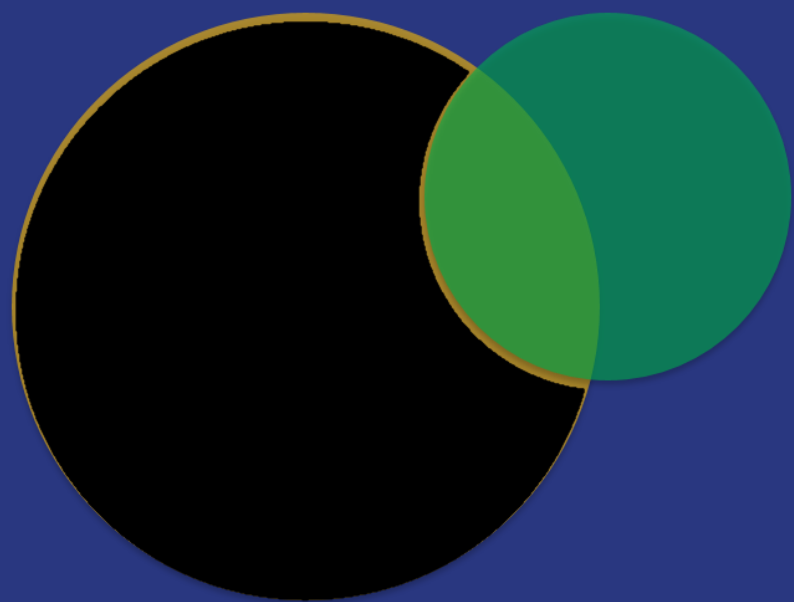
聯集



交集



差集



LAB

使用 Union 與 Intersect

在 LinqSamples 加入新專案

- 方案名稱： LinqSamples
- 專案名稱： LinqSample013
- 範本：Console Application
- csproj nullable disable

直接在 Main Method 加入 程式碼

```
static void Main(string[] args)
{
    var list1 = new List<int> { 1, 2, 3, 4, 5, 6 };
    var list2 = new List<int> { 1, 3, 4, 7, 8, 9 };
    var union = list1.Union(list2);
    Console.WriteLine("聯集的結果為 :");
    foreach (var item in union )
    {
        Console.WriteLine(item);
    }

    var intersect = list1.Intersect(list2);
    Console.WriteLine("交集的結果為 :");
    foreach (var item in intersect)
    {
        Console.WriteLine(item);
    }

    Console.ReadLine();
}
```

執行

討論

- 了解 Union ,
Intersect 的意義和其使用方式了嗎？
- 如果蓋上電腦，你是否能算出兩個集合的聯集與交集？



LAB

使用 Except

在 LinqSamples 加入新專案

- 方案名稱： LinqSamples
- 專案名稱： LinqSample014
- 範本：Console Application
- csproj nullable disable

直接在 Main Method 加入 程式碼

```
static void Main(string[] args)
{
    var list1 = new List<int> { 1, 2, 3, 4, 5, 6 };
    var list2 = new List<int> { 1, 3, 4, 7, 8, 9 };
    var aEXb = list1.Except(list2);
    Console.WriteLine("A 差集 B 的結果為 :");
    foreach (var item in aEXb)
    {
        Console.WriteLine(item);
    }

    var bEXa = list2.Except(list1);
    Console.WriteLine("B 差集 A 的結果為: ");
    foreach (var item in bEXa)
    {
        Console.WriteLine(item);
    }

    Console.ReadLine();
}
```




執行

討論

- 了解 Except 的意義和其使用方式了嗎？
- 如果蓋上電腦，你是否能算出兩個集合的差集？



猜猜這兩行在做甚麼？

```
list1.Except(list2).Union(list2.Except(list1));
```

```
list1.Union(list2).Except(list1.Intersect(list2));
```

Distinct

- 排除重複，如果有兩個以上相同的資料，只會取一個

LAB

使用 Distinct

在 LinqSamples 加入新專案

- 方案名稱： LinqSamples
- 專案名稱： LinqSample015
- 範本：Console Application
- csproj nullable disable

直接在 Main Method 加入 程式碼

```
static void Main(string[] args)
{
    var list =
        new List<string> { "台北", "台北", "洛杉磯", "紐約", "紐約", "台北" };
    var result = list.Distinct();
    foreach (var item in result)
    {
        Console.WriteLine(item);
    }

    Console.ReadLine();
}
```



執行

Skip 與 Take

- Skip
 - 跳過幾筆
- Take
 - 取得幾筆

LAB

使用 Skip 與 Take

在 LinqSamples 加入新專案

- 方案名稱： LinqSamples
- 專案名稱： LinqSample016
- 範本：Console Application
- csproj nullable disable

```
static void Main(string[] args)
{
    var list = new List<string> {"A","B","C","D","E","F","F"};
    var resultOfSkip = list.Skip(3);
    Console.WriteLine("Skip(3) 的結果 ");
    Display(resultOfSkip);

    var resultOfTake = list.Take(3);
    Console.WriteLine("Take(3) 的結果 ");
    Display(resultOfTake);

    var resultOfSkipTake = list.Skip(2).Take(2);
    Console.WriteLine("Skip(2).Take(2) 的結果 ");
    Display(resultOfSkipTake);

    Console.ReadLine();
}

static void Display(IEnumerable<string> source)
{
    foreach (var item in source)
    {
        Console.WriteLine(item);
    }
}
```

執行

複製成另一個集合

- `ToArray`
- `ToList`
- `ToDictionary`

LAB

使用 ToArray、ToList 與 ToDictionary

在 LinqSamples 加入新專案

- 方案名稱： LinqSamples
- 專案名稱： LinqSample017
- 範本：Console Application
- csproj nullable disable

加入 MyData Class，並建立其內容

```
internal class MyData
{
    public string Name
    { get; set; }

    public int Age
    { get; set; }
}
```

在 Program Class 加入產生 List<MyData> 的方法

```
static List<MyData> CreateList()
{
    /* 使用 集合運算式 */
    return
    [
        new() { Name = "Bill", Age = 47 },
        new() { Name = "John", Age = 37 },
        new() { Name = "Tom", Age = 48 },
        new() { Name = "David", Age = 36 },
    ];
}
```

C#12.0 的集合運算式

在 Main Method 加入 程式碼

```
static void Main(string[] args)
{
    var list = CreateList();
    var result1 = list.Where((x) => x.Age > 40).ToList();
    var result2 = list.Where((x) => x.Age > 40).ToArray();
    // 使用 Name 當群組分類的索引鍵，而值資料仍然是 MyData
    var result3 = list.Where((x) => x.Age > 40).ToDictionary((x) => x.Name);

    foreach (var item in result3 )
    {
        Console.WriteLine(item.Key);
        Console.WriteLine($"{item.Value.Name} -- {item.Value.Age}");
    }
    Console.WriteLine("-----");

    // 使用 Name 當群組分類的索引鍵，而且用 Age 當值資料
    var result4 = list.ToDictionary((x) => x.Name, (y) => y.Age);
    foreach (var item in result4)
    {
        Console.WriteLine(item.Key);
        Console.WriteLine(item.Value);
    }
    Console.ReadLine();
}
```



執行

討論

- ToList、ToArray 和 ToDictionary 的用法



群組

- **GroupBy**

- 依據條件將資料分成群組
- 回傳型別是 `IEnumerable<IGrouping<TKey,TSource>>`

LAB

GroupBy – Method Expression

在 LinqSamples 加入新專案

- 方案名稱： LinqSamples
- 專案名稱： LinqSample018
- 範本：Console Application
- csproj nullable disable

加入 MyData Class，並建立其內容

```
internal class MyData
{
    public string City
    { get; set; }

    public string Name
    { get; set; }
}
```

在 Program Class 加入產生 List<MyData> 的方法

```
static List<MyData> CreateList()
{
    /* 使用 集合運算式 */
    return
    [
        new() { Name = "Bill", City = "台北" },
        new() { Name = "John", City = "台北" },
        new() { Name = "Tom", City = "高雄" },
        new() { Name = "David", City = "台南" },
        new() { Name = "Jeff", City = "台南" },
    ];
}
```

C#12.0 的集合運算式

在 Main Method 加入 程式碼

```
static void Main(string[] args)
{
    var list = CreateList();
    var result = list.GroupBy((x) => x.City);
    foreach (var item in result)
    {
        Console.WriteLine($"住在 : {item.Key}");
        foreach (var p in item)
        {
            Console.WriteLine(p.Name);
        }
        Console.WriteLine("-----");
    }
    Console.ReadLine();
}
```



執行

LAB

GroupBy – Query Expression

在 LinqSamples 加入新專案

- 方案名稱： LinqSamples
- 專案名稱： LinqSample019
- 範本：Console Application
- csproj nullable disable

加入 MyData Class，並建立其內容

```
internal class MyData
{
    public string City
    { get; set; }

    public string Name
    { get; set; }
}
```

在 Program Class 加入產生 List<MyData> 的方法

```
static List<MyData> CreateList()
{
    /* 使用 集合運算式 */
    return
    [
        new() { Name = "Bill", City = "台北" },
        new() { Name = "John", City = "台北" },
        new() { Name = "Tom", City = "高雄" },
        new() { Name = "David", City = "台南" },
        new() { Name = "Jeff", City = "台南" },
    ];
}
```

C#12.0 的集合運算式

在 Main Method 加入 程式碼

```
static void Main(string[] args)
{
    var list = CreateList();
    var result =
        from o in list
        group o by o.City into gp
        select gp;

    foreach (var item in result)
    {
        Console.WriteLine($"住在 : {item.Key}");
        foreach (var p in item)
        {
            Console.WriteLine(p.Name);
        }
        Console.WriteLine("-----");
    }
    Console.ReadLine();
}
```



執行

討論

- 解釋 GroupBy 的用法



關聯

- **Join**

- 根據相符索引鍵的兩個序列的項目相互關聯
- Join 通常用 Query Expression 寫

班級	老師
1A	Bill
1B	David

班級	學生
1A	魯夫
1A	索隆
1B	櫻木
1A	香吉士
1B	流川楓



班級	老師	學生
1A	Bill	魯夫
1A	Bill	索隆
1A	Bill	香吉士
1B	David	櫻木
1B	David	流川楓

LAB

Join

在 LinqSamples 加入新專案

- 方案名稱： LinqSamples
- 專案名稱： LinqSample020
- 範本：Console Application
- csproj nullable disable

加入 TeacherInfo Class , StudentInfo Class和 ResultInfo Class , 並建立其內容

```
internal class TeacherInfo
{
    public string ClassName { get; set; }
    public string Teacher { get; set; }
}
```

```
internal class StudentInfo
{
    public string ClassName { get; set; }
    public string Student { get; set; }
}
```

```
internal class ResultInfo
{
    public string ClassName { get; set; }
    public string Teacher { get; set; }
    public string Student { get; set; }
}
```

在 Program Class 加入產生來源資料的方法

```
static List<TeacherInfo> CreateTeachers()
{
    return new List<TeacherInfo>()
    {
        new TeacherInfo { ClassName = "1A" , Teacher = "Bill" },
        new TeacherInfo { ClassName = "1B" , Teacher = "David"}
    };
}
```

```
static List<StudentInfo> CreateStudents()
{
    return new List<StudentInfo>()
    {
        new StudentInfo { ClassName = "1A" , Student = "魯夫" },
        new StudentInfo { ClassName = "1A" , Student = "索隆" },
        new StudentInfo { ClassName = "1B" , Student = "櫻木" },
        new StudentInfo { ClassName = "1A" , Student = "香吉士"},
        new StudentInfo { ClassName = "1B" , Student = "流川楓"}
    };
}
```

回到集合初始化設定式寫法

在 Main Method 加入 程式碼

```
static void Main(string[] args)
{
    var teachers = CreateTeachers();
    var students = CreateStudents();
    var result =
        from t in teachers
        join s in students
        on t.ClassName equals s.ClassName
        select
            new ResultInfo
            { ClassName = t.ClassName, Teacher = t.Teacher, Student = s.Student };

    foreach (var item in result)
    {
        Console.WriteLine($"{item.ClassName} : {item.Teacher} : {item.Student}");
    }

    Console.ReadLine();
}
```



執行

討論

- 解釋 Join 的用法



排序

- **OrderBy**
 - 依照條件由小到大排序
- **OrderByDescending**
 - 依照條件由大到小排序
- **ThenBy**
 - 在 Method Expression 需要兩個條件排序使用
- **ThenByDescending**
 - 在 Method Expression 需要兩個條件排序使用

LAB

OrderBy -- Method Expression

在 LinqSamples 加入新專案

- 方案名稱： LinqSamples
- 專案名稱： LinqSample021
- 範本：Console Application
- csproj nullable disable

加入 MyData Class，並建立其內容

```
internal class MyData
{
    public string Name
    { get; set; }

    public int Age
    { get; set; }
}
```

在 Program Calss 加入產生 List<MyData> 的方法

```
static List<MyData> CreateList()
{
    return new List<MyData>()
    {
        new MyData { Name = "Bill" , Age = 47 },
        new MyData { Name = "John" , Age = 37 },
        new MyData { Name = "Tom" , Age = 48 },
        new MyData { Name = "David", Age = 36 },
        new MyData { Name = "Bill" , Age = 35 },
    };
}
```

回到集合初始化設定式寫法

在 Program Class 加入 程式碼

```
static void Main(string[] args)
{
    var list = CreateList();
    var order1 = list.OrderBy((x) => x.Age);
    Display(order1);
    var order2 = list.OrderByDescending((x) => x.Age);
    Display(order2);
    var order3 = list.OrderBy((x) => x.Name).ThenBy((x) => x.Age);
    Display(order3);
    var order4 = list.OrderBy((x) => x.Name).ThenByDescending((x) => x.Age);
    Display(order4);
    Console.ReadLine();
}

static void Display(IOrderedEnumerable<MyData> source)
{
    foreach (var item in source)
    {
        Console.WriteLine($"{item.Name} : {item.Age}");
    }
    Console.WriteLine("-----");
}
```



執行

LAB

OrderBy -- Query Expression

在 LinqSamples 加入新專案

- 方案名稱： LinqSamples
- 專案名稱： LinqSample022
- 範本：Console Application
- csproj nullable disable

加入 MyData Class，並建立其內容

```
internal class MyData
{
    public string Name
    { get; set; }

    public int Age
    { get; set; }
}
```

在 Program Calss 加入產生 List<MyData> 的方法

```
static List<MyData> CreateList()
{
    return new List<MyData>()
    {
        new MyData { Name = "Bill" , Age = 47 },
        new MyData { Name = "John" , Age = 37 },
        new MyData { Name = "Tom" , Age = 48 },
        new MyData { Name = "David", Age = 36 },
        new MyData { Name = "Bill" , Age = 35 },
    };
}
```

回到集合初始化設定式寫法

在 Program Class 加入 程式碼

```
static void Main(string[] args)
{
    var list = CreateList();
    var order1 =
        from o in list
        orderby o.Name, o.Age
        select o;
    Display(order1);
    var order2 =
        from o in list
        orderby o.Name descending, o.Age descending
        select o;
    Display(order2);
    Console.ReadLine();
}

static void Display(IOrderedEnumerable<MyData> source)
{
    foreach (var item in source)
    {
        Console.WriteLine($"{item.Name} : {item.Age}");
    }
    Console.WriteLine("-----");
}
```



執行

討論

- 解釋排序的用法



linq 的方法
是可以混在一起用的

在這裏你將學到

Learn How to Learn

- 學新東西、新技術的能力
- 尋找解答的能力
- 隨時吸取新知識的能力

跟著你一輩子的能力 ...